

VELLORE INSTITUTE OF TECHNOLOGY BHOPAL

VITyarthi Project Report

LOCAL TRANSPORT HELPER

An AI & ML Based Local Transportation Assistance System

Submitted as part of the VITyarthi Build Your Own Project

Student Name	Pratim Ghosh
Registration No.	25BAI11606
Programme	B.Tech CSE AI & ML
Semester	1
College	Vellore Institute of Technology Bhopal

Academic Project Report

2026

CERTIFICATE

This is to certify that **Pratim Ghosh**, Registration No. **25BAI11606**, a student of **B.Tech CSE AI & ML** at **Vellore Institute of Technology Bhopal**, has prepared the project report entitled “**Local Transport Helper**” as part of the VITyarthi Build Your Own Project activity.

The project has been documented with emphasis on problem understanding, functional requirements, system design, implementation approach, testing, results, challenges, learnings and future enhancements.

Project Guide / Faculty	_____
Signature	_____
Date	_____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Vellore Institute of Technology Bhopal for providing the opportunity to undertake this VITyarthi project. The project has helped me understand how concepts from Artificial Intelligence and Machine Learning can be organized into a practical software solution.

I am thankful to the faculty members and project mentors for their guidance, feedback and encouragement throughout the planning and documentation of the project. I also acknowledge the importance of the VITyarthi project guidelines in providing a structured approach to problem identification, design, implementation and evaluation.

Finally, I thank my family and friends for their support and encouragement during the development of this project.

TABLE OF CONTENTS

No.	Section
1	Introduction
2	Problem Statement
3	Objectives
4	Functional Requirements
5	Non-Functional Requirements
6	System Architecture
7	Design Diagrams
8	Dataset and Machine Learning Approach
9	Design Decisions & Rationale
10	Implementation Details
11	Screenshots / Results
12	Testing Approach
13	Challenges Faced
14	Learnings & Key Takeaways
15	Future Enhancements
16	Conclusion
17	References

1. INTRODUCTION

Local transportation is an important part of everyday travel, especially for students and other users who need short-distance connectivity. Choosing between different transport options can involve considering travel distance, estimated time, expected fare and convenience. When these factors are considered manually, the decision can become slow or inconsistent.

Local Transport Helper is proposed as an AI and ML oriented assistance system that organizes transport-related inputs and produces a useful recommendation. The system is designed around a simple user interaction: the user provides travel information and preferences, the system processes the information, estimates relevant transport values, compares alternatives and presents a recommendation.

The project is intentionally modular. The major modules are user/location input, transport recommendation, fare/time prediction, transport comparison and result presentation. This structure supports maintainability and provides clear separation between data processing, prediction and user-facing output.

2. PROBLEM STATEMENT

Local commuters often need to select an appropriate mode of transport without having a simple tool that compares the available alternatives using the same criteria. Manual estimation may not provide a consistent view of expected fare and travel time.

Problem: Design and implement a local transport assistance system that accepts travel information, processes relevant transport attributes, estimates fare and/or travel time using a machine-learning based approach, compares transport alternatives and provides an understandable recommendation.

Scope

- Short-distance and local transportation assistance.
- Comparison of multiple transport categories such as bus, auto, taxi/cab and e-rickshaw.
- Prediction/estimation of fare and travel time from available input features.
- Simple recommendation based on predicted and user-provided factors.
- Demonstration through a modular software implementation.

Target Users

- Students and young commuters.
- Local residents and short-distance travellers.
- Users who want a quick comparison of cost and time.

3. OBJECTIVES

- Develop a practical AI/ML-oriented solution for local transport assistance.
- Provide a clear input and output workflow for transport queries.
- Create a prediction component for estimated fare and/or travel time.
- Compare multiple local transport options using consistent criteria.

- Generate a recommendation that is understandable to a non-technical user.
- Use modular program organization so that individual components can be tested and improved.
- Document the design, implementation, testing and evaluation according to the VITyarthi requirements.

4. FUNCTIONAL REQUIREMENTS

Module	Input	Output
User & Location Input	Source, destination, distance, preference	Validated travel query
Data Processing	Raw transport/travel attributes	Processed feature set
Fare/Time Prediction	Processed features	Estimated fare and/or travel time
Transport Recommendation	Predictions + preferences	Recommended option
Transport Comparison	Multiple option records	Comparison by cost/time
Results & Analytics	Prediction and recommendation data	Readable result summary

4.1 User Interaction Workflow

1. User enters source and destination or relevant travel parameters.
2. The system validates the input.
3. The system prepares the features required for prediction.
4. The ML component estimates fare and/or travel time.
5. Available transport alternatives are compared.
6. A suitable transport option is recommended.
7. The final result is displayed to the user.

5. NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Performance	The system should process a normal transport query within a reasonable response time.
Usability	Inputs and results should be simple, readable and understandable.
Reliability	Invalid or incomplete input should be handled without crashing the application.
Maintainability	Modules should be separated so individual components can be updated independently.
Scalability	The design should allow additional transport types and features to be added later.
Error Handling	Validation should identify missing, invalid or out-of-range values.

6. SYSTEM ARCHITECTURE

The proposed architecture separates the application into input, processing, ML prediction, recommendation and presentation layers.

Layer	Responsibility
User Interface / Input	Collects travel details and preferences.
Validation Layer	Checks required fields and input ranges.
Data Processing Layer	Cleans, transforms and prepares features.
ML Prediction Layer	Uses the trained model to estimate fare/time.
Recommendation Layer	Ranks or selects transport alternatives.
Presentation Layer	Displays comparison, prediction and recommendation.

Architecture Flow

User Input → Validation → Feature Processing → ML Prediction → Transport Comparison → Recommendation → Result Display

The architecture supports the VITYarathi requirement for proper architectural design and modular implementation. It also allows the ML model to be replaced without redesigning the complete user interaction layer.

7. DESIGN DIAGRAMS

7.1 Use Case Diagram

Actor: User

User → Enter Travel Details

User → Select Transport Preference

User → Request Prediction

User → Compare Transport Options

User → View Recommendation

Use-case relationships: Travel Query → Validate Input → Predict → Compare → Recommend → Display.

7.2 Workflow Diagram

Start → Enter source/destination → Enter travel parameters → Validate input → Prepare features → Predict fare/time → Compare options → Generate recommendation → Display results → **End**

7.3 Sequence Diagram

User → **Interface**: Submit travel query

Interface → **Validator**: Validate input

Validator → **Processor**: Send valid data

Processor → **ML Model**: Request prediction

ML Model → **Processor**: Return estimated values

Processor → **Recommendation Engine**: Rank options

Recommendation Engine → **Interface**: Return recommendation

Interface → **User**: Display results

7.4 Class / Component Diagram

Component	Main Responsibility
TransportQuery	Stores source, destination and user preferences.
DataProcessor	Preprocesses and transforms input features.
PredictionModel	Loads/trains model and produces predictions.
TransportOption	Represents an available transport alternative.
RecommendationEngine	Compares options and selects a recommendation.
ResultFormatter	Prepares prediction and comparison output.
Validator	Checks input and handles invalid values.

7.5 Data / Storage Design

If persistent storage is used, a transport record can contain fields such as `record_id`, `transport_type`, `distance`, `estimated_time`, `estimated_fare` and other features required by the model. The storage design is kept simple so the project can focus on the AI/ML workflow.

8. DATASET AND MACHINE LEARNING APPROACH

8.1 Dataset Description

The ML component requires a dataset containing transport-related observations. A suitable record can include transport type, distance, traffic or time-related attributes where available, and a target such as fare or travel time. The dataset is divided into input features and target values.

Important: The final implementation should document the actual dataset used, its source, number of records, feature names and any preprocessing performed. Those implementation-specific values are not supplied in the VITYarthi guideline document and therefore are not invented in this report.

8.2 Data Preprocessing

- Remove or handle missing values.
- Convert categorical transport types into numerical representations.
- Check numerical ranges and inconsistent records.
- Separate input features from the target variable.
- Split the dataset into training and testing subsets.

8.3 Model Selection Rationale

For a tabular prediction problem, a supervised regression model is appropriate when the target is a continuous value such as fare or travel time. A baseline model can be established first, followed by a tree-based regression model if non-linear relationships are present. The final model should be selected using validation performance and error metrics rather than model complexity alone.

8.4 Evaluation Methodology

Regression performance can be evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and, where appropriate, coefficient of determination (R^2). Lower MAE/RMSE indicates smaller prediction error, while a higher R^2 indicates that more variance in

the target is explained by the model.

9. DESIGN DECISIONS & RATIONALE

Decision	Rationale
Modular architecture	Makes the project easier to understand, test and maintain.
Separate prediction layer	Allows the ML model to be changed without redesigning the interface.
Transport comparison	Provides more useful output than a single prediction.
Input validation	Reduces errors caused by incomplete or invalid user data.
Regression approach	Suitable for continuous fare/time estimation.
Simple user output	Keeps the result useful for non-technical users.

10. IMPLEMENTATION DETAILS

10.1 Suggested Project Structure

local-transport-helper/

- app.py
- data_preprocessing.py
- train_model.py
- predict.py
- recommendation.py
- transport_data.csv
- model.pkl
- utils.py
- test_prediction.py
- README.md

10.2 Input Validation

Validation checks should ensure that required values are present and numerical inputs such as distance are within sensible ranges. Invalid values should result in a clear message rather than an application failure.

10.3 Prediction Pipeline

The prediction pipeline consists of feature preparation, loading the trained model, generating the prediction and formatting the output. The same preprocessing logic used during training should be applied during prediction to avoid inconsistent feature representations.

10.4 Recommendation Logic

The recommendation layer combines predicted values and user preferences. For example, a cost-focused user may prefer a lower-fare option, while a time-focused user may prefer a faster option. A weighted scoring strategy can be used so the decision is transparent and adjustable.

10.5 Error Handling

- Missing source/destination or required parameters.

- Non-numeric distance or invalid numerical values.
- Unavailable model file or invalid model state.
- Unexpected dataset values.
- Empty prediction or comparison results.

11. SCREENSHOTS / RESULTS

The final submission should include screenshots of the implemented application. Recommended screenshots are listed below. Actual screenshots should be inserted after the application has been executed.

Figure	Recommended Screenshot
Figure 1	Main Local Transport Helper interface
Figure 2	Travel input form with source and destination
Figure 3	Transport prediction result
Figure 4	Transport comparison output
Figure 5	Final recommendation
Figure 6	ML evaluation/results screen or console output

Result Interpretation

A successful result should show that the system accepts a valid transport query, processes the required features, produces a prediction, compares available options and presents a recommendation. Model metrics should be reported using the actual values obtained during experimentation.

12. TESTING APPROACH

Test ID	Test Case	Expected Result
T01	Valid travel input	Query is accepted and processed.
T02	Missing required field	Validation message is displayed.
T03	Invalid distance value	Invalid input is rejected.
T04	Prediction request	Model returns an estimated value.
T05	Multiple transport options	Options are compared.
T06	Recommendation generation	Suitable option is displayed.
T07	Model unavailable	Application handles the error gracefully.
T08	Empty comparison result	User receives a meaningful message.

Testing Strategy

Testing combines input validation tests, functional tests for individual modules and model evaluation using a held-out test set. The purpose is to verify both the software workflow and the predictive component.

13. CHALLENGES FACED

- Converting a general transportation problem into measurable input and output variables.
- Preparing consistent tabular data for machine-learning use.
- Handling categorical transport types and numerical travel features together.
- Choosing a model that provides useful predictions without unnecessary complexity.
- Designing recommendation logic that remains understandable to the user.
- Handling invalid or incomplete user input safely.
- Presenting ML results in a way that is useful rather than overly technical.

14. LEARNINGS & KEY TAKEAWAYS

- Understanding how a real-world problem can be converted into a software specification.
- Applying preprocessing and supervised learning concepts to a practical prediction task.
- Understanding the importance of selecting features and targets carefully.
- Learning why model evaluation is necessary before using predictions.
- Understanding modular architecture and separation of responsibilities.
- Improving documentation through diagrams, requirements and testing tables.
- Recognizing the importance of validation and error handling in practical applications.

15. FUTURE ENHANCEMENTS

- Integrate live map and route information through an appropriate mapping service.
- Add real-time traffic information.
- Support additional cities and local transport categories.
- Improve prediction accuracy using larger and more representative datasets.
- Add personalized recommendations based on user priorities.
- Add historical travel analytics.
- Provide mobile-friendly deployment.
- Add explainable recommendations showing why a transport option was selected.

16. CONCLUSION

Local Transport Helper demonstrates how Artificial Intelligence and Machine Learning concepts can be applied to a practical local transportation problem. The proposed system combines structured input, data processing, prediction, transport comparison and recommendation into one modular workflow.

The project is aligned with the VITyarthi requirement to identify a meaningful problem, design a technical solution, implement subject concepts and demonstrate understanding through documentation and evaluation. The modular design also provides a foundation for future enhancements such as live routing,

traffic-aware prediction and personalized transport recommendations.

17. REFERENCES

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (provided project guideline document).
2. Python documentation – language and standard-library concepts used for implementation.
3. Scikit-learn documentation – supervised learning, preprocessing and model evaluation concepts.
4. Pandas documentation – tabular data processing and analysis concepts.
5. Matplotlib documentation – visualization concepts, where applicable.

APPENDIX A – VITyarthi REQUIREMENT ALIGNMENT

The following checklist maps the report to the requirements in the supplied VITyarthi guideline.

VITyarthi Requirement	Report Coverage
Three major functional modules	Sections 4 and 10
Clear input/output structure	Section 4
Logical workflow	Sections 4 and 7
Four or more non-functional requirements	Section 5
Architecture design	Section 6
Use Case / Workflow / Sequence / Class design	Section 7
Dataset / model / evaluation for ML	Section 8
Implementation details	Section 10
Screenshots / results	Section 11
Testing	Section 12
Challenges / Learnings / Future Enhancements	Sections 13–15
References	Section 17